Семинар 8. CRDT для последовательностей и CALM-теорема

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Третье домашнее задание – автопроверка (10 баллов = 2 + 2 + 4 + 2)
- Проставлю баллы по зеленым галочкам в GitHub Classroom

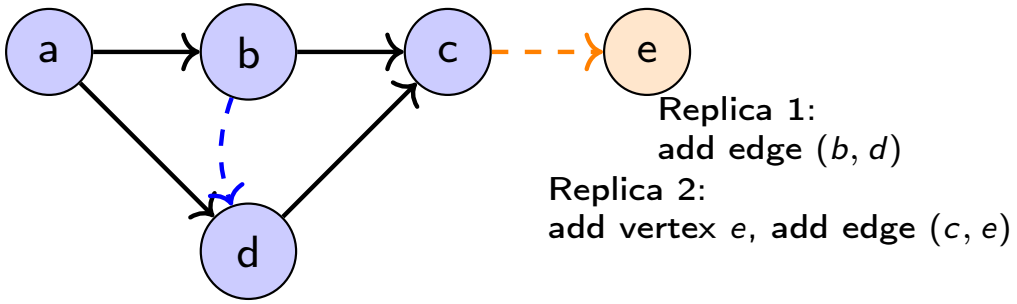
Содержание

1. Другие структуры данных
2. Немного про последовательности
3. CALM-теорема
4. Итоги

Add-only Monotonic DAG

CRDT-граф: добавление вершин и рёбер, без удаления. Реализуется как два G-Set: V (вершины), E (рёбра).

Свойство монотонности DAG: при добавлении ребра (u, v) — предикат проверяется *локально*; merge может нарушить DAG-условие (отсутствие циклов), поэтому часто используют **только partial order** как инвариант.



Применение: системы контроля версий (commit graph), workflow engines, causality tracking.

Causal Tree и 2P-Set Graph

Causal Tree (CT)

- Каждая операция — узел дерева с ссылкой на **причину** (parent op)
- Структура: $\{(id, \text{parent_id}, \text{payload})\}$
- Базис для WOOT, RGA, Treedoc
- Tombstone: удалённые узлы маркируются, но не удаляются
- Исп. в Atom Teletype, xi-editor

2P-Set Graph

- Два 2P-Set: V (вершины), E (рёбра)
- `addVertex`, `addEdge`, `removeVertex`, `removeEdge`
- Инвариант: $e = (u, v) \in E \Rightarrow u \in V \wedge v \in V$
- Поддержание инварианта при merge — сложность
- Shapiro et al. (2011) предлагают **vertex-removal precondition**

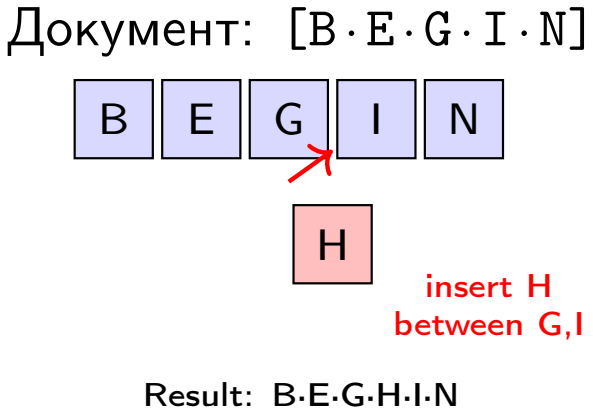
Содержание

1. Другие структуры данных
2. Немного про последовательности
3. CALM-теорема
4. Итоги

WOOT (Without Operational Transformation)

Каждый символ:
$\langle id, v, W_L, W_R, \text{visible} \rangle$

- id — уникальный идентификатор (сайт, clock)
- W_L, W_R — соседи *в момент вставки*
- При merge: вставляем символ, используя W_L, W_R для определения позиции
- Конкурентные вставки в одну позицию разрешаются детерминированным tiebreaking по id

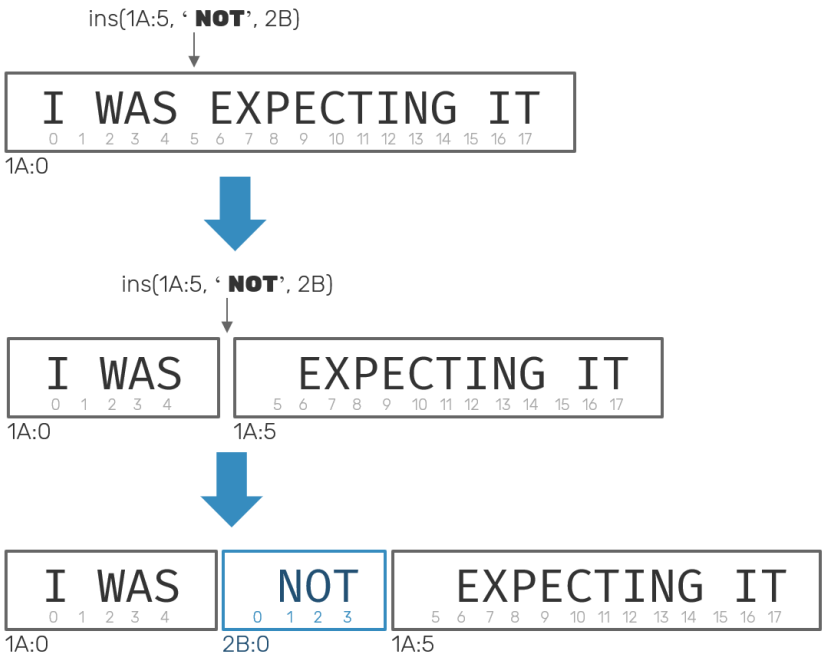


Сложность: $O(n^2)$ на вставку в худшем случае (много конкурентных вставок).

RGA (Replicated Growable Array)

Идея: связный список с уникальными s4vector (s, sum, sid, seq)

- Каждый элемент: $\langle s4, v, \text{next}, \text{deleted} \rangle$
- Вставка: находим predecessor по s4, вставляем после него
- При конфликте (два insert после одного и того же): сортировка по s4 (timestamp desc)
- Tombstone для удалений (элемент остаётся в списке, маркируется)



Применения: Riak DT, Automerge (ранняя версия), Yjs (частично).
Сложность: $O(n)$ для вставки, $O(1)$ амортизированного для конвергентных вставок.

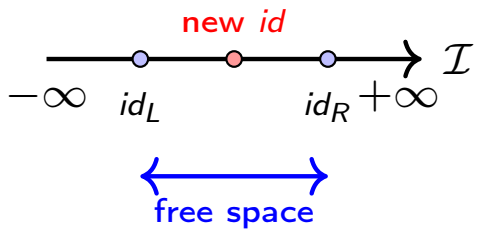
Logoot и LSEQ

Logoot: каждому элементу присваивается **уникальный позиционный идентификатор** из упорядоченного пространства $\mathcal{I}$.

- $id = [(p_1, s_1), (p_2, s_2), \ldots]$ — список пар (позиция, сайт)
- Вставка между id_L и id_R: выбрать id с $id_L < id < id_R$
- Документ = упорядоченное множество (id, v) пар
- Удаление: просто удалить по id

LSEQ: адаптивное выделение идентификаторов для контроля длины пути — решает проблему «надувания» id в Logoot.

Идентификаторный интервал

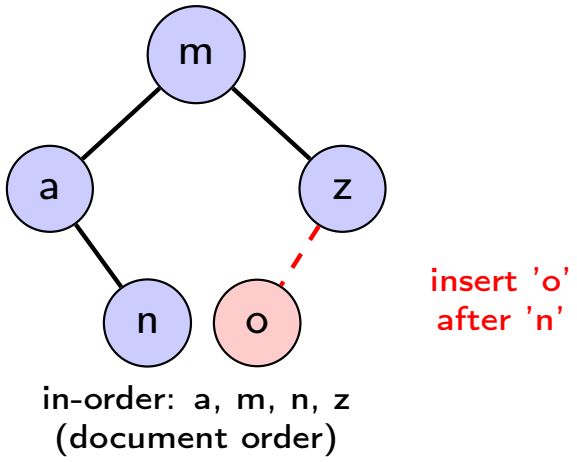


LSEQ: адаптивно чередует стратегии boundary+ / boundary- для balance дерева

Treedoc

Идея: последовательность как бинарное дерево с *in-order traversal*.

- Узел дерева = элемент + уникальный *id* (путь в дереве)
- Идентификатор пути естественно упорядочен: in-order traversal
- Вставка между L и R: добавить узел как левый потомок следующего или правый потомок текущего
- **Disambiguator** (site id) для конкурентных вставок в один узел (mini-tree)



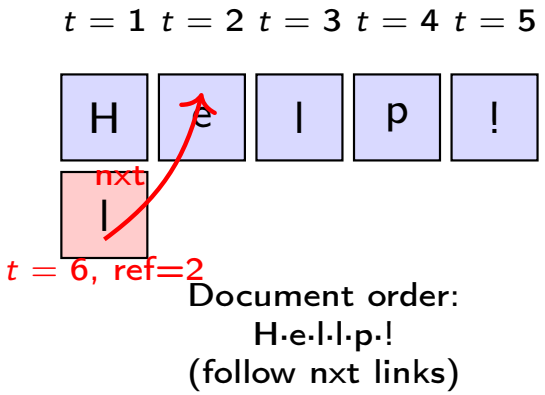
Компакция: tombstone-узлы без потомков удаляются при synchronization — уникальность CRDT!

Chronofold

Ключевые идеи:

- Хранить операции в **chrono-order** (timestamp), не в document order
- **Implicit linked list:** поддерживается массив «следующих» указателей $nxt[\cdot]$
- Document order восстанавливается обходом implicit list за $O(n)$
- Вставка новой операции: $O(1)$ амортизированно при FIFO доставке

Chrono order (массив)



Fugue и FugueMax

Мотивация:
Существующие CRDT для последовательностей (RGA, YATA, Logoot) не имеют **формальных гарантий качества порядка** при конкурентных вставках. Вводится понятие **non-interleaving**.

FugueMax:
Расширение с явным понятием «maximal non-interleaving» — оптимально для UX.

A вставляет "foo"

B вставляет "bar"

Fugue:
"foobar" или "barfoo"
(не "fbaoor")

RGA может дать:
"fboaor" ← плохо!

Non-interleaving

Пусть пользователи A и B конкурентно вставляют последовательности символов $a_1 \ldots a_k$ и $b_1 \ldots b_m$ в одну позицию. CRDT *не перемежает* их, если итоговый порядок — либо $a_1 \ldots a_k b_1 \ldots b_m$, либо $b_1 \ldots b_m a_1 \ldots a_k$.

Результат: RGA, Logoot, LSEQ — *нарушают* non-interleaving. Fugue — **гарантирует**.

Содержание

1. Другие структуры данных
2. Немного про последовательности
3. CALM-теорема
4. Итоги

CALM-теорема: Consistency As Logical Monotonicity

Основная идея (Hellerstein et al., 2010):
Программа достигает консистентности без координации $\Leftrightarrow$ её логика *монотонна*.

Monotonic logic
(order-insensitive)

Convergent
without coordination

Non-monotonic logic
(order-sensitive)

Needs coordination

Содержание

1. Другие структуры данных
2. Немного про последовательности
3. CALM-теорема
4. Итоги

Открытые проблемы

Открытые вопросы

- Delta-CRDT (меньше трафика, Almeida et al., 2018)
- Pure op-based CRDT (Baquero et al., 2017)
- CRDT в БД: Electric SQL, Ditto, Automerge 2.0

Итоги

- Выдано 3-е домашнее задание про CRDT.